

# LAB MANUAL OF DATA STRUCTURES AND ALGORITHMS (DSA)



**Name : Syed Affan Hussain**

**Roll No. : 2020-CS-180**

**Sec : D**

**Semester : 3<sup>rd</sup>**

**Submitted to Miss Javeria**

**COMPUTER SCIENCE DEPARTMENT**  
**SIR SYED UNIVERSITY OF ENGINEERING  
AND TECHNOLOGY, KARACHI**

## **Data Structures and Algorithms (CS-212)**

### **Spring Semester 2021**

#### **I. Lab Objectives**

In the laboratory, Visual Studio 2017 will be used to implement the concepts of Data Structures (including arrays, stacks, queues and linked lists), advanced data structures (including trees and graphs). Simple searching and sorting algorithms (linear and binary search, selection and insertion sort).

#### **II. Lab Outcomes**

Upon the completion of Data Structures practical, Student will be able to describe the usage of various data structures and operations for maintaining data structures. Design and implement various data structures and algorithms.

#### **III. TEXT BOOK:**

- Drozdek, Adam. Data Structures and Algorithms in C++, 4th edition, USA, Published by Cengage Learning 2013.

#### **IV. REFERENCE BOOKS:**

- Weiss, Mark. Data Structures and Algorithm Analysis in C++, 4<sup>th</sup> edition, USA, Published by Pearson 2014.

**Table of Contents**

| <b>Lab No.</b> | <b>Topic</b>                                                                                                                                                                                         |
|----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1              | An overview of Data Structures and their Operations: Brief Revision of Array<br>Implementation of Array operations One Dimensional Array<br>Multidimensional Array Abstract data types.              |
| 2              | Algorithm Analysis: Implementation of Pseudo coding, Control Structures<br>Programming examples for Analysis of Algorithms, Efficiency of Algorithms,<br>Complexity of Algorithms                    |
| 3              | Pointers: Implementation of Null pointer, Void Pointer, Invalid Pointer,<br>Dangling Pointer Reference Variable                                                                                      |
| 4              | Sorting and Searching Techniques: Implementation for elementary sorting<br>algorithms such as Bubble Sort, Selection Sort, Insertion Sort, Quick Sort,<br>Merge Sort                                 |
| 5              | Implementation of searching algorithms such as Linear Search and Binary<br>Search                                                                                                                    |
| 6              | Linked Lists: Implementation of linked list with the help of algorithms for<br>Insertion, Deletion and Search an element Implementation of Doubly Link List<br>Implementation of Circular Link List. |
| 7              | Stacks: An array and link list based implementation of STACK                                                                                                                                         |
| 8              | Queues: An array and link list based implementation of QUEUES                                                                                                                                        |
| 9              | Trees: Implementation of Binary Trees, Array/Link List representation of<br>binary trees in memory                                                                                                   |
| 10             | Tree Traversal (Binary Trees, Pre, In, and Post-order traversal)<br><br>Binary Search Trees: Implementation of Binary Search Trees Searching,                                                        |
| 11             | Inserting, and Deleting in Binary Search Trees, Binary Heaps, Heap creation<br>and operations                                                                                                        |
| 12             | Graphs: Implementation of Directed, Undirected Graphs, Sequential<br>representation of graphs, Operations on Graphs                                                                                  |
| 13             | Graph Traversals (Depth-First Traversal, Breadth First Traversal)                                                                                                                                    |

# Lab # 1

## Getting Started (Revision of C++)

### I. Introduction to:

1. Recall all programming techniques
2. Use of Array, Structures, Functions & Pointers

Basic If-Else Statement

There are three types of control statements used in C language

- If else statements
- Switch statements
- Goto statements

```
if ( <condition> )
{
    // Execute these statements if <condition> is TRUE
}
else if ( <another condition> )
{
    // Execute these statements if <another condition> is TRUE and
    // <condition> is FALSE
}
```

Example:

```
include<iostream>
using namespace std;

int main()                // Most important part of the program!
{
    int age;               // Need a variable...

    cout<<"Please input your age: "; // Asks for age
    cin>> age;              // The input is put in age
    cin.ignore();          // Throw away enter
    if ( age < 100 )
    {
        // If the age is less than 100
        cout<<"You are pretty young!\n"; // Just to show you it works.
    }
    else if (age == 100)
    {
        // I use else just to show an example
        cout<<"You are old\n";      // Just to show you it works...
    }
    else
    {
```

```

        cout<<"You are really old\n"; // Executed if no other statement
is
    }
    cin.get();
}

```

## **II. Array**

An array is a container object that holds a fixed number of values of a single type. The length of an array is established when the array is created. After creation, its length is fixed. You have seen an example of arrays already, in the main method of the "Hello World!" application.

Example

```

#include <iostream>
using namespace std;

void printarray (int arg[], int length)
{
    for (int n=0; n<length; ++n)
        cout<<arg[n] << ' ';
        cout<< '\n';
}

int main ()
{
    Int firstarray[] = {5, 10, 15};
    Int secondarray[] = {2, 4, 6, 8, 10};
    printarray(firstarray,3);
    printarray(secondarray,5);
}

```

## **III. Structures**

A data structure is a group of data elements grouped together under one name. These data elements, known as members, can have different types and different lengths

Definition Of Structure

```

struct<struct-type>
{
    <type><identifier_list>;
    <type><identifier_list>;
    .....
};

```

### **Example**

```

#include <iostream>
using namespace std;

```

```

struct
person {
    charname[50];
    int age;
    float salary;
};

int main()
{
    person p1;
    cout<< "Enter Full name: ";
    cin.get(p1.name, 50);
    cout<< "Enter age: ";
    cin>> p1.age;
    cout<< "Enter salary: ";
    cin>> p1.salary;

    cout<< "\n Displaying Information." << endl;
    cout<< "Name: " << p1.name <<endl;
    cout<<"Age: " << p1.age <<endl;
    cout<< "Salary: " << p1.salary;

    return 0;
}

```

## **OUTPUT**

```

Enter Full name: Magdalena Dankova
Enter age: 27
Enter salary: 1024.4

```

```

Displaying Information.
Name: Magdalena Dankova
Age: 27
Salary: 1024.4

```

#### **IV. Pointer to Structures**

A pointer variable can be created not only for native types like (int, float, double etc.) but they can also be created for user defined types like structure. Pointer to Structure

##### **Example:**

```
#include<iostream>
using namespace std;

struct Distance
{
    int feet;
    float inch;
};

int main()
{
    Distance *ptr, d;
    ptr = &d;
    cout<< "Enter feet: ";
    cin>> (*ptr).feet;
    cout<< "Enter inch: ";
    cin>> (*ptr).inch;

    cout<< "Displaying information." <<endl;
    cout<< "Distance = " << (*ptr).feet << " feet " << (*ptr).inch << " inches";

    return 0;
}
```

##### **OUTPUT**

```
Enter feet: 4
Enter inch: 3.5
Displaying information.
Distance = 4 feet 3.5 inches
```

In this program, a pointer variable ptr and normal variable d of type structure Distance is defined. The address of variable d is stored to pointer variable, that is, ptr is pointing to variable d. Then the member function of variable d is accessed using pointer.

Note: Since pointer ptr is pointing to variable d in this program, (\*ptr).inch and d.inch is exact same cell. Similarly, (\*ptr).feet and d.feet is exact same cell.

The syntax to access member function using pointer is ugly and there is alternative notation -> which is more common.

```
ptr->feet is same as (*ptr).feet
ptr->inch is same as (*ptr).inch
```

## Lab Tasks

1. Write a Program to enter three integers and output the smallest integer using IF.

### PROGRAM

```
#include<iostream>
#include<conio.h>

using namespace std;

int main()
{
    int a, b, c;
    cout << "\n \t Enter First Number : ";
    cin >> a;
    cout << "\n \t Enter Second Number : ";
    cin >> b;
    cout << "\n \t Enter Third Number : ";
    cin >> c;
    cout << "\n \t_____ " << endl;
    if (a < b && a < c)
    {
        cout << " \t " << a << " is the smallest element " << endl;
    }
    else if (b < a && b < c)
    {
        cout << " \t " << b << " is the smallest element " << endl;
    }
    else
    {
        cout << " \t " << c << " is the smallest element " << endl;
    }
    cout << " \t_____ " << endl;

    system("pause");
    return 0;
}
```

### OUTPUT

```
Enter First Number : 45
Enter Second Number : 15
Enter Third Number : 35

_____
15 is the smallest element
_____
Press any key to continue . . .
```



2. Write a Program to enter 10 integers in a single-dimension array and then print out the array in ascending order.

### **PROGRAM**

```
int main()
{
    int arr[10], t;
    cout << "\n \t Enter 10 numbers & Then We print out the Array in Ascending Order" << endl;
    cout << "\n \t _____" << endl;
    for (int i = 0; i < 10; i++)
    {
        cout << " \t \t Enter " << i + 1 << " Number : ";
        cin >> arr[i];
    }
    cout << " \t _____" << endl;

    for (int j = 0; j < 9; j++)
    {
        for (int k = 0; k < 9; k++)
        {
            if (arr[k] > arr[k + 1])
            {
                t = arr[k + 1];
                arr[k + 1] = arr[k];
                arr[k] = t;
            }
        }
    }
    cout << "\n \t _____" << endl;
    cout << "\n \t Array in Ascending Order : ";
    for (int i = 0; i < 10; i++)
    {
        cout << arr[i] << ", ";
    }
    cout << endl;
    cout << " \t _____" << endl;
    return 0;
}
```

## OUTPUT

```
Enter 10 numbers & Then We print out the Array in Ascending Order

Enter 1 Number : 7
Enter 2 Number : 9
Enter 3 Number : 5
Enter 4 Number : 8
Enter 5 Number : 4
Enter 6 Number : 6
Enter 7 Number : 2
Enter 8 Number : 1
Enter 9 Number : 3
Enter 10 Number : 15

Array in Ascending Order : 1, 2, 3, 4, 5, 6, 7, 8, 9, 15,
```

3. Write a program to create structure named student. Take information of student from user as input (StdID, StdName, StdAge etc.) Display the output.

## PROGRAM

```
int main()
{
    cout << "\n \t Enter First student Detail : " << endl;
    student s1;
    cout << " \t Enter His Id : ";
    cin >> s1.student_id;
    cout << " \t Enter His Name : ";
    cin >> s1.student_name;
    cout << " \t Enter His Age : ";
    cin >> s1.age;

    cout << "\n \t Enter Second student Detail : " << endl;
    student s2;
    cout << " \t Enter His Id : ";
    cin >> s2.student_id;
    cout << " \t Enter His Name : ";
    cin >> s2.student_name;
    cout << " \t Enter His Age : ";
    cin >> s2.age;

    cout << "\n _____ " << endl;
    cout << "\t First Student Details :) " << endl;
    cout << "\t Age : " << s1.age << endl;
    cout << "\t Name : " << s1.student_name << endl;
    cout << "\t ID : " << s1.student_id << endl;
```

```

cout << "\n _____ " << endl;
cout << "\t Second Student Details :) " << endl;
cout << "\t Age : " << s2.age << endl;
cout << "\t Name : " << s2.student_name << endl;
cout << "\t ID : " << s2.student_id << endl;
cout << " _____ " << endl;
system("pause");
return 0;
}

```

## OUTPUT

```

Enter First student Detail :
Enter His Id : 4518
Enter His Name : Affan
Enter His Age : 20

Enter Second student Detail :
Enter His Id : 7845
Enter His Name : Asad
Enter His Age : 22

-----
First Student Details :>
Age : 20
Name : Affan
ID : 4518

-----
Second Student Details :>
Age : 22
Name : Asad
ID : 7845

-----
Press any key to continue . . .

```

4. Perform above task using pointers to structure

## PROGRAM

```

struct student
{
    int student_id;
    string student_name;
    int age;
};
int main()
{
    cout << "\n \t Enter student Detail : " << endl;
    student* ptr, s;
    ptr = &s;
    cout << " \t Enter His Id : ";
    cin >> (*ptr).student_id;
    cout << " \t Enter His Name : ";
    cin >> (*ptr).student_name;
    cout << " \t Enter His Age : ";
}

```

```

cin >> (*ptr).age;

cout << "\n _____" << endl;
cout << "\t First Student Details :) " << endl;
cout << "\t Age : " << (*ptr).age << endl;
cout << "\t Name : " << (*ptr).student_name << endl;
cout << "\t ID : " << (*ptr).student_id << endl;
cout << "\n _____" << endl;

system("pause");
return 0;
}

```

## OUTPUT

```

Enter student Detail :
Enter His Id : 4518
Enter His Name : Affan
Enter His Age : 20

First Student Details :)
Age : 20
Name : Affan
ID : 4518

Press any key to continue . . .

```

## Lab # 2

# Measurement of Time Complexity of an algorithm

Sometimes, there are more than one way to solve a problem. We need to learn how to compare the performance different algorithms and choose the best one to solve a particular problem. While analyzing an algorithm, we mostly consider time complexity and space complexity. Time complexity of an algorithm quantifies the amount of time taken by an algorithm to run as a function of the length of the input. Similarly, Space complexity of an algorithm quantifies the amount of space or memory taken by an algorithm to run as a function of the length of the input.

Time and space complexity depends on lots of things like hardware, operating system, processors, etc. However, we don't consider any of these factors while analyzing the algorithm. We will only consider the execution time of an algorithm.

Let's start with a simple example. Suppose you are given an array A and an integer x and you have to find if x exists in array A.

Simple solution to this problem is traverse the whole array A and check if the any element is equal to x.

```
For i : 1 to length  
of A   if A[i] is  
equal to x  
return TRUE  
return FALSE
```

Each of the operation in computer take approximately constant time. Let each operation takes c time. The number of lines of code executed is actually depends on the value of x. During analyses of algorithm, mostly we will consider worst case scenario, i.e., when x is not present in the array A. In the worst case, the **if** condition will run N times where N is the length of the array A. So, in the worst case, total execution time will be  $(Nc+c)$ .  $N*c$  for the **if** condition and c for the **return** statement (ignoring some operations like assignment of i).

As we can see that the total time depends on the length of the array A. If the length of the array will increase the time of execution will also increase.

**I. Order of growth** is how the time of execution depends on the length of the input. In the above example, we can clearly see that the time of execution is linearly depends on the length of the array. Order of growth will help us to compute the running time with ease. We will ignore the lower order terms, since the lower order terms are relatively insignificant for large input. We use different notation to describe limiting behavior of a function.

### **a) O-notation:**

To denote asymptotic upper bound, we use O-notation. For a given function g(n), we denote by  $O(g(n))$  (pronounced "big-oh of g of n") the set of functions:

$O(g(n)) = \{ f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ such that } 0 \leq f(n) \leq c g(n) \text{ for all } n \geq n_0 \}$

### ***b) $\Omega$ -notation:***

To denote asymptotic lower bound, we use  $\Omega$ -notation. For a given function  $g(n)$ , we denote by  $\Omega(g(n))$  (pronounced “big-omega of g of n”) the set of functions:

$$\Omega(g(n)) = \{ f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ such that } 0 \leq c \cdot g(n) \leq f(n) \text{ for all } n \geq n_0 \}$$

### ***c) $\Theta$ -notation:***

To denote asymptotic tight bound, we use  $\Theta$ -notation. For a given function  $g(n)$ , we denote by  $\Theta(g(n))$  (pronounced “big-theta of g of n”) the set of functions:

$$\Theta(g(n)) = \{ f(n) : \text{there exist positive constants } c_1, c_2 \text{ and } n_0 \text{ such that } 0 \leq c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n) \text{ for all } n \geq n_0 \}$$

## **II. Time complexity notations**

While analyzing an algorithm, we mostly consider O-notation because it will give us an upper limit of the execution time i.e. the execution time in the worst case.

To compute O-notation we will ignore the lower order terms, since the lower order terms are relatively insignificant for large input.

Let  $f(N) = 2 \cdot N^2 + 3 \cdot N + 5$

$$O(f(N)) = O(2 \cdot N^2 + 3 \cdot N + 5) = O(N^2)$$

Let's consider some example:

1.

```
int count = 0; for
(int i = 0; i < N;
i++)
for (int j = 0; j < i;
j++)    count++;
```

Lets see how many times **count++** will run.

When  $i=0$ , it will run 0 times.

When  $i=1$ , it will run 1 times.

When  $i=2$ , it will run 2 times and so on.

Total number of times **count++** will run is  $0+1+2+\dots+(N-1) = \frac{N(N-1)}{2}$ . So the time complexity will be  $O(N^2)$ .

2.

```
int count = 0;
for (int i = N; i > 0; i /= 2)
    for (int j = 0; j < i; j++)
        count++;
```

This is a tricky case. In the first look, it seems like the complexity is  $O(N \cdot \log N)$ .  $N$  for the  $j$ 's loop and  $\log N$  for  $i$ 's loop. But its wrong. Lets see why.

Think about how many times **count++** will run.

When  $i=N$ , it will run  $N$  times.

When  $i=N/2$ , it will run  $N/2$  times.

When  $i=N/4$ , it will run  $N/4$  times and so on.

Total number of times **count++** will run is  $N+N/2+N/4+\dots+1=2N$ . So the time complexity will be  $O(N)$ .

## Lab # 3

### Pointers: Implementation of Null pointer, Void Pointer, Invalid Pointer, Dangling Pointer Reference Variable

A **pointer** is a variable whose value is the address of another variable. Like any variable or constant, you must declare a pointer before you can work with it. The general form of a pointer variable declaration is –

type \*var-name;

Here, **type** is the pointer's base type; it must be a valid C++ type and **var-name** is the name of the pointer variable. The asterisk you used to declare a pointer is the same asterisk that you use for multiplication. However, in this statement the asterisk is being used to designate a variable as a pointer. Following are the valid pointer declaration –

```
int *ip;      // pointer to an integer
double *dp;   // pointer to a double
float *fp;    // pointer to a float
char *ch      // pointer to character
```

The actual data type of the value of all pointers, whether integer, float, character, or otherwise, is the same, a long hexadecimal number that represents a memory address. The only difference between pointers of different data types is the data type of the variable or constant that the pointer points to.

#### I. Null Pointers

The NULL pointer is a constant with a value of zero defined in several standard libraries, including iostream. Consider the following program

```
#include <iostream>
using namespace std;
int main ()
{
    int *ptr = NULL;
    cout << "The value of ptr is " << ptr ;

    return 0;
}
```

When the above code is compiled and executed, it produces the following result –

The value of ptr is 0

On most of the operating systems, programs are not permitted to access memory at address 0 because that memory is reserved by the operating system. However, the memory address 0 has



special significance; it signals that the pointer is not intended to point to an accessible memory location. But by convention, if a pointer contains the null (zero) value, it is assumed to point to nothing.

To check for a null pointer you can use an if statement as follows –

```
if(ptr)    // succeeds if p is not null
if(!ptr)   // succeeds if p is null
```

Thus, if all unused pointers are given the null value and you avoid the use of a null pointer, you can avoid the accidental misuse of an uninitialized pointer. Many times, uninitialized variables hold some junk values and it becomes difficult to debug the program.

## **II. Pointers vs Arrays**

Pointers and arrays are strongly related. In fact, pointers and arrays are interchangeable in many cases. For example, a pointer that points to the beginning of an array can access that array by using either pointer arithmetic or array-style indexing. Consider the following program –

```
#include <iostream>
using namespace std;
const int MAX = 3;
int main ()
{
    int var[MAX] = {10, 100, 200};
    int *ptr;    // let us have array address in pointer.
    ptr = var;

    for (int i = 0; i < MAX; i++)
    {
        cout << "Address of var[" << i << "] = ";
        cout << ptr << endl;

        cout << "Value of var[" << i << "] = ";
        cout << *ptr << endl;    // point to the next location
        ptr++;
    }
    return 0;
}
```

When the above code is compiled and executed, it produces result something as follows –

Address of var[0] = 0xbfa088b0  
Value of var[0] = 10

Address of var[1] = 0xbfa088b4  
Value of var[1] = 100

Address of var[2] = 0xbfa088b8  
Value of var[2] = 200

### III. Pointer to Pointer

A pointer to a pointer is a form of multiple indirections or a chain of pointers. Normally, a pointer contains the address of a variable. When we define a pointer to a pointer, the first pointer contains the address of the second pointer, which points to the location that contains the actual value as shown below.

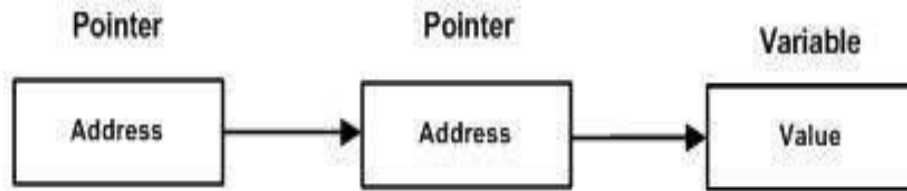


Fig. 1 Pointer to Pointer

A variable that is a pointer to a pointer must be declared as such. This is done by placing an additional asterisk in front of its name. For example, following is the declaration to declare a pointer to a pointer of type int – `int **var;`

When a target value is indirectly pointed to by a pointer to a pointer, accessing that value requires that the asterisk operator be applied twice, as is shown below in the example –

```

#include <iostream>
using namespace std;
int main ()
{
    int var;
    int *ptr;
    int **pptr;
    var = 3000; // take the address of var
    ptr = &var; // take the address of ptr using address of operator &
    pptr = &ptr; // take the value using pptr
    cout << "Value of var :" << var << endl;
    cout << "Value available at *ptr :" << *ptr << endl;
    cout << "Value available at **pptr :" << **pptr << endl;

    return 0;
}
  
```

When the above code is compiled and executed, it produces the following result –

```

Value of var : 3000
Value available at *ptr : 3000
Value available at **pptr : 3000
  
```

## TASK

Write a function that returns a pointer to the maximum value of an array, if array is empty return NULL.

### Coding :

```
#include<iostream>
#include<conio.h>
using namespace std;
int *Maximum(int *a, int size)
{
    if (size == 0)
    {
        cout << "\n \t Array is Empty !" << endl;
        return NULL;
    }
    else
    {
        int max_val = a[0];
        int *max_pos=0;
        for (int i = 0; i < size; i++)
        {
            if (a[i] > max_val)
            {
                max_val = a[i];
                max_pos = &a[i];
            }
        }
        return max_pos;
    }
}
int main()
{
    int size, *n;
    int num[10];
    cout << "\n \t Enter the size of array : ";
    cin >> size;

    for (int i = 0; i < size; i++)
    {
        cout << "\n \t Enter " << i + 1 << " Element : ";
        cin >> num[i];
    }
    n = Maximum(num, size);
    cout << "\n \t_____ " << endl;
    cout << " \t The maximum value of an array is : " << *n << endl;
    cout << " \t_____ " << endl;
    system("pause");
    return 0;
}
```

**Output :**

```
Enter the size of array : 4
```

```
Enter 1 Element : 5
```

```
Enter 2 Element : 8
```

```
Enter 3 Element : 15
```

```
Enter 4 Element : 6
```

---

```
The maximum value of an array is : 15
```

---

```
Press any key to continue . . . _
```

```
Enter the size of array : 0
```

```
Array is Empty ?
```

---



# Lab # 4

## Implementation for elementary sorting algorithms

### **I. Introduction to:**

1. Sorting a static list of values using different Algorithms
2. Performance issue of sorting algorithms

### **II. Bubble Sort**

Bubble sort is a simple and well-known sorting algorithm. It is used in practice once in a blue moon and its main application is to make an introduction to the sorting algorithms. Bubble sort belongs to  $O(n^2)$  sorting algorithms, which makes it quite inefficient for sorting large data volumes. Bubble sort is stable and adaptive.

#### **a) Algorithm**

1. Compare each pair of adjacent elements from the beginning of an array and, if they are in reversed order, swap them.
2. If at least one swap has been done, repeat step 1.

You can imagine that on every step big bubbles float to the surface and stay there. At the step, when no bubble moves, sorting stops. Let us see an example of sorting an array to make the idea of bubble sort clearer.

Example. Sort {5, 1, 12, -5, 16} using bubble sort.

#### **b) Complexity analysis**

Average and worst case complexity of bubble sort is  $O(n^2)$ . Also, it makes  $O(n^2)$  swaps in the worst case. Bubble sort is adaptive. It means that for almost sorted array it gives  $O(n)$  estimation. Avoid implementations, which don't check if the array is already sorted on every step (any swaps made). This check is necessary, in order to preserve adaptive property

### **III. Selection Sort**

Selection sort is one of the  $O(n^2)$  sorting algorithms, which makes it quite inefficient for sorting large data volumes. Selection sort is notable for its programming simplicity and it can over perform other sorts in certain situations (see complexity analysis for more details).

#### **a) Algorithm**

The idea of algorithm is quite simple. Array is imaginary divided into two parts - sorted one and unsorted one. At the beginning, sorted part is empty, while unsorted one contains whole

array. At every step, algorithm finds minimal element in the unsorted part and adds it to the end of the sorted one. When unsorted part becomes empty, algorithm stops.

When algorithm sorts an array, it swaps first element of unsorted part with minimal element and then it is included to the sorted part. This implementation of selection sort is not stable. In case of linked list is sorted, and, instead of swaps, minimal element is linked to the unsorted part, selection sort is stable.

Let us see an example of sorting an array to make the idea of selection sort clearer.

Example. Sort {5, 1, 12, -5, 16, 2, 12, 14} using selection sort.

## **b) Complexity analysis**

Selection sort stops, when unsorted part becomes empty. As we know, on every step number of unsorted elements decreased by one. Therefore, selection sort makes  $n$  steps ( $n$  is number of elements in array) of outer loop, before stop. Every step of outer loop requires finding minimum in unsorted part. Summing up,  $n + (n - 1) + (n - 2) + \dots + 1$ , results in  $O(n^2)$  number of comparisons. Number of swaps may vary from zero (in case of sorted array) to  $n - 1$  (in case array was sorted in reversed order), which results in  $O(n)$  number of swaps. Overall algorithm complexity is  $O(n^2)$ .

Fact, that selection sort requires  $n - 1$  number of swaps at most, makes it very efficient in situations, when write operation is significantly more expensive, than read operation.

## **IV. Insertion Sort**

Insertion sort belongs to the  $O(n^2)$  sorting algorithms. Unlike many sorting algorithms with quadratic complexity, it is actually applied in practice for sorting small arrays of data. For instance, it is used to improve quicksort routine. Some sources notice, that people use same algorithm ordering items, for example, hand of cards.

### **a) Algorithm**

Insertion sort algorithm somewhat resembles selection sort. Array is imaginary divided into two parts - sorted one and unsorted one. At the beginning, sorted part contains first element of the array and unsorted one contains the rest. At every step, algorithm takes first element in the unsorted part and inserts it to the right place of the sorted one. When unsorted part becomes empty, algorithm stops. Sketchy, insertion sort algorithm step looks like this:

Becomes

The idea of the sketch was originally posted [here](#).

Let us see an example of insertion sort routine to make the idea of algorithm clearer.

Example. Sort {7, -5, 2, 16, 4} using insertion sort.

The ideas of insertion

The main operation of the algorithm is insertion. The task is to insert a value into the sorted part of the array. Let us see the variants of how we can do it.

"Sifting down" using swaps

## Lab Task

Create a program that take an array of 10 inputs from the user and generate the sorted out put using the following Algorithms;

- Bubble Sort
- Insertion Sort
- Selection Sort

## PROGRAM

```
#include<iostream>
#include<conio.h>
#include<stdio.h>

using namespace std;

void bubbleSort(int arr[], int n)
{
    int i, j, temp;
    cout << "\n \t Before Sorted : ";
    for (int i = 0; i < n; i++)
    {
        cout << arr[i] << ", ";
    }
    for (i = 0; i < n; i++)
    {
        for (j = 0; j < n - i - 1; j++) //number of passes
        {
            if (arr[j] > arr[j + 1])
            {
                // swap the elements
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
        // print the sorted array
        cout << "\n \t After Sorted : ";
        for (i = 0; i < n; i++)
        {
            cout << arr[i] << ", ";
        }
        cout << endl;
    }
}

void Insertion_sort(int arr[], int length)
{
x:
    cout << "\n \t Before Sorted : ";
    for (int i = 0; i < length; i++)
```





```

        cout << "\n \t_____ " << endl;
        for (i = 0; i < n; i++)
        {
            cout << " \t \t Enter element no. " << i + 1 << " : ";
            cin >> arr[i];
        }
        cout << " \t_____ " << endl;
jump:

        cout << "\n \t Enter 1 for Bubble Sort " << endl;
        cout << "\n \t Enter 2 for Insert Sort " << endl;
        cout << "\n \t Enter 3 for Selection Sort " << endl;
        cout << "\n \t Enter 4 for Exit " << endl;
        cout << "\n \t_____ " << endl;
        cout << " \t \t Enter the Position : ";
        cin >> call;

        cout << "\n \t_____ " <<
endl;
        switch (call)
        {
            case(1):
            {
                bubbleSort(arr, n);
                break;
            }
            case(2):
            {
                Insertion_sort(arr, n);
                break;
            }
            case(3):
            {
                Selection_sort(arr, n);
                break;
            }
            case(4):
            {
                exit(0);
            }
        }
        cout << "\n \t_____ " <<
endl;
        cout << endl;
        goto jump;
        system("pause");
        return 0;
    }

```

## OUTPUT

```
Enter the number of elements to be sorted : 10
```

```
Enter element no. 1 : 99
Enter element no. 2 : 88
Enter element no. 3 : 77
Enter element no. 4 : 66
Enter element no. 5 : 55
Enter element no. 6 : 44
Enter element no. 7 : 33
Enter element no. 8 : 22
Enter element no. 9 : 11
Enter element no. 10 : 01
```

```
Enter 1 for Bubble Sort
```

```
Enter 2 for Insert Sort
```

```
Enter 3 for Selection Sort
```

```
Enter 4 for Exit
```

```
Enter the Position : 1
```

```
Before Sorted : 99, 88, 77, 66, 55, 44, 33, 22, 11, 1,
After Sorted : 1, 11, 22, 33, 44, 55, 66, 77, 88, 99,
```

```
Enter the Position : 2
```

```
Before Sorted : 1, 11, 22, 33, 44, 55, 66, 77, 88, 99,
After Sorted : 1, 11, 22, 33, 44, 55, 66, 77, 88, 99,
```

```
Enter the Position : 3
```

```
Before Sorted : 1, 11, 22, 33, 44, 55, 66, 77, 88, 99,
After Sorted : 1, 11, 22, 33, 44, 55, 66, 77, 88, 99,
```

```
Enter the Position : 4
```

```
\Users\innoc\source\repos\DSA_4\Debug\DSA_4.exe (process 1196) exited with code 0.
  automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatic
  when debugging stops.
  Press any key to close this window . . .
```

# **Lab # 5**

## **Implementation of searching algorithms over an Array based List**

### **I. Introduction to:**

1. Different searching techniques using static Data Structure i.e., Array
2. Efficiencies and deficiencies of both the searching techniques
3. Internals and Abstraction of matrices and their representation

### **II. Binary Search**

Binary Search is applied on the sorted array or list. In binary search, we first compare the value with the elements in the middle position of the array. If the value is matched, then we return the value. If the value is less than the middle element, then it must lie in the lower half of the array and if it's greater than the element then it must lie in the upper half of the array. We repeat this procedure on the lower (or upper) half of the array. Binary Search is useful when there are large numbers of elements in an array.

#### **a) Binary Search Algorithm**

Step 1: Set the list to be the whole list

Step 2: Find the middle value of the list

Step 3: If the middle value is equal to the target then we declare victory and stop

Step 4: If the middle item is less than the target, then we set the new list to be the upper half of the old list and we repeat from Step 2 using the new list

Step 5: If the middle value is greater than the target, then we set the new list to be the bottom half of the list, and we repeat from Step 2 with the new list.

### **III. Linear Search**

A linear search is the basic and simple search algorithm. A linear search searches an element or value from an array till the desired element or value is not found and it searches in a sequence order. It compares the element with all the other elements given in the list and if the element is matched it returns the value index else it returns -1. Linear Search is applied on the unsorted or unordered list when there are fewer elements in a list.

#### **a) Linear Search Algorithm**

Step1: Start with the first item in the list

Step 2: Compare the current item to the target

Step 3: If the current value matches the target then we declare victory and stop

Step 4: If the current value is less than the target then set the current item to be the next item and repeat from Step 2

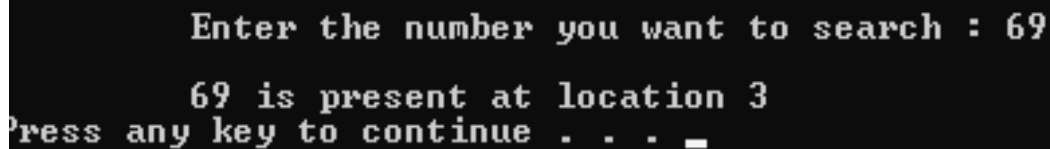
## Lab Tasks

1. Translate the linear search algorithm into a program which either find the location LOC where ITEM appears in ARRAY or return LOC=0

### PROGRAM :

```
#include <iostream>
#include<conio.h>
using namespace std;
int main()
{
    int num, c, n=6;
    int arr[] = {12, 35, 69, 74, 165, 54};
    cout << "\n \t Enter the number you want to search : ";
    cin >> num;
    for (c = 0; c < n; c++)
    {
        if (arr[c] == num)
        {
            cout << "\n \t " << num << " is present at location "<< c+1;
            break;
        }
    }
    if (c == n)
    {
        cout << "\n \t " << num << " isn't present in the array. \n";
    }
    cout << endl;
    system("pause");
    return 0;
}
```

### OUTPUT :

A screenshot of a terminal window with a black background and white text. The text shows the program's execution: it prompts for a number to search, the user enters 69, the program finds it at location 3, and then prompts for a key press to continue.

```
Enter the number you want to search : 69
69 is present at location 3
Press any key to continue . . . _
```

2. Translate binary search and insertion algorithm into a program which finds either the location LOC where ITEM appears in ARRAY or the location LOC where ITEM should be inserted into ARRAY. (For object 3, 4, and 5 take an array of 10 elements given below)

## PROGRAM :

```
#include<iostream>
#include<conio.h>
using namespace std;
void i_sort(int a[], int n)
{
    int j, key;
    for (int i = 1; i < n; i++)
    {
        j = i;
        while (j > 0 && a[j - 1] > a[j])
        {
            key = a[j];
            a[j] = a[j - 1];
            a[j - 1] = key;
            j--;
        }
    }
    cout << "\n-----\n";
    cout << " after sorted array : \n";
    for (int i = 0; i < n; i++)
    {
        cout << "\n the index value of " << i << " is : " << a[i] << "\t";
    }
    cout << "\n-----\n";
}
int main()
{
    int a[100], n, m, num, insert, add;
    int mid, start, end;
    char choice;
    cout << "\n enter the size of array : ";
    cin >> n;
    cout << "\n-----\n";
    for (int i = 0; i < n; i++)
    {
        cout << " enter the index value " << i << " : ";
        cin >> a[i];
    }
    cout << "\n-----\n";
    cout << "\n \t You want to add more array value through insertion algorithm
(y/n) :";
    cin >> choice;
    switch (choice)
    {
        case 'y':
        {
            cout << "\n \t How many value you want to add : ";
            cin >> add;
            for (int j = 0; j < add; j++)
            {
                cout << "\n-----\n";
                cout << "\n \t Enter the number you want to add : ";
```

```

        cin >> insert;
        cout << "\n-----\n";

        while (n >= n + 1)
        {
            a[n + 1] = a[n];
            n--;
        }
        a[n] = insert;
        n++;
    }
    cout << "\n-----\n";
    cout << " after adding value in array : ";
    cout << "\n-----\n";
    for (int i = 0; i < n; i++)
    {
        cout << "\n \t The index value of " << i << " is : " << a[i] <<
endl;
    }
    cout << "\n-----\n";
    break;
}
case 'n':
{
    break;
}
}
i_sort(a, n);
cout << "\n \t Enter the number you want to search : ";
cin >> num;
start = 0;
end = n - 1;
mid = (start + end) / 2;
while (a[mid] != num && start <= end)
{
    if (num > a[mid])
    {
        start = mid + 1;
    }
    else
    {
        end = mid - 1;
    }
    mid = (start + end) / 2;
}
if (start <= end)
{
    cout << "\n-----\n";
    cout << num << " is present at index '" << mid << "' in the array " <<
endl;
    cout << "\n-----\n";
}
else
{
    cout << "\n-----\n";
    cout << num << " is not present in the array....." << endl;
    cout << "\n-----\n";
}
system("pause");
return 0;
}

```

## OUTPUT :

```
enter the size of array : 5
-----
enter the index value 0 : 8
enter the index value 1 : 4
enter the index value 2 : 2
enter the index value 3 : 15
enter the index value 4 : 7
-----
      You want to add more array value through insertion algorithm (y/n) : n
-----
after sorted array :
the index value of 0 is : 2
the index value of 1 is : 4
the index value of 2 is : 7
the index value of 3 is : 8
the index value of 4 is : 15
-----
      Enter the number you want to search : 15
-----
15 is present at index '4' in the array
```

3. Write a program which uses Binary Search to search the elements 52, and 33 and print it.

## PROGRAM :

```
#include<iostream>
#include<conio.h>
using namespace std;
int binarySearch(int arr[], int l, int r, int num)
{
    if (l <= r)
    {
        int mid = (l + r) / 2;
        if (arr[mid] == num)
        {
            return mid;
        }
        if (arr[mid] > num)
        {
            return binarySearch(arr, l, mid - 1, num);
        }
        if (arr[mid] < num)
        {
            return binarySearch(arr, mid + 1, r, num);
        }
    }
    return -1;
}
int main()
{
    int arr[10] = { 1, 3, 7, 15, 18, 20, 33, 36, 40, 52 };
    int n = sizeof(arr) / sizeof(arr[0]);
    int times, a[50];
    cout << "\n \t How many times you want to search : ";
    cin >> times;
    for (int i = 0; i < times; i++)
    {
        cout << "\n \t _____ " << endl;
```



```

        cout << "\n \t Enter the " << i + 1 << " value : ";
        cin >> a[i];
        int index = binarySearch(arr, 0, n - 1, a[i]);
        if (index == -1)
        {
            cout << "\n \t " << a[i] << " is not present in the array." <<
endl;
        }
        else
        {
            cout << "\n \t " << a[i] << " is present at index " << index << "
in the array." << endl;
        }
    }
    cout << "\n \t _____ " << endl;

    system("pause");
    return 0;
}

```

## OUTPUT :

```

How many times you want to search : 2
_____

Enter the 1 value : 33
33 is present at index 6 in the array.
_____

Enter the 2 value : 52
52 is present at index 9 in the array.
_____

ss any key to continue . . . _

```

4. Write a program which uses Binary Search to search the elements 45, and 78 and delete it.  
 NUMBERS = [11,22,33,36,45,52,57,60,64,78]

## PROGRAM :

```

#include<iostream>
#include<conio.h>
using namespace std;
int main()
{
    int a[100], n, m, num;
    int mid, start, end;
    cout << "\n enter the size of array : ";
    cin >> n;
    cout << "\n-----\n";
    for (int i = 0; i < n; i++)
    {
        cout << " enter the index value " << i << " : ";
    }
}

```

```

        cin >> a[i];
    }
    cout << "\n-----\n";
    cout << " how many number you want to search : ";
    cin >> m;
    for (int j = 0; j < m; j++)
    {
        cout << "\n enter the number you want to search : ";
        cin >> num;
        start = 0;
        end = n - 1;
        mid = (start + end) / 2;
        while (a[mid] != num && start <= end)
        {
            if (num > a[mid])
            {
                start = mid + 1;
            }
            else
            {
                end = mid - 1;
            }
            mid = (start + end) / 2;
        }
        if (start <= end)
        {
            cout << "\n-----\n";
            cout << num << " is present at index '" << mid << "' in the array\n";
            cout << "\n-----\n";
        }
        else
        {
            cout << "\n-----\n";
            cout << num << " is not present in the array....." << endl;
            cout << "\n-----\n";
        }
        while (mid <= n)
        {
            a[mid] = a[mid + 1];
            mid++;
        }
        n--;
        cout << "\n-----\n";
        cout << " after deleting the array value that we found above : ";
        cout << "\n-----\n";
        for (int i = 0; i < n; i++)
        {
            cout << " the index value of " << i << " is : " << a[i] << endl;
        }
        cout << "\n-----\n";
    }
    system("pause");
    return 0;
}

```

## OUTPUT :

```
enter the size of array : 5
-----
enter the index value 0 : 4
enter the index value 1 : 6
enter the index value 2 : 8
enter the index value 3 : 10
enter the index value 4 : 12
-----
how many number you want to search : 2
enter the number you want to search : 6
-----
6 is present at index '1' in the array
-----
```

```
-----
enter the number you want to search : 12
-----
12 is present at index '3' in the array
-----
after deleting the array value that we found above :
-----
the index value of 0 is : 4
the index value of 1 is : 8
the index value of 2 is : 10
-----
Press any key to continue . . .
```

## **Lab # 6**

# **Implementation of linked list with the help of algorithms for Insertion, Deletion and Search an element**

### **I. Introduction to:**

- LINED LIST
- Implementation of LINKED LIST
- Operation of LINKED LIST

### **II. Singly-linked list**

Linked list is a very important dynamic data structure. Basically, there are two types of linked list, singly-linked list and doubly-linked list. In a singly-linked list every element contains some data and a link to the next element, which allows keeping the structure. On the other hand, every node in a doubly-linked list also contains a link to the previous node. Linked list can be an underlying data structure to implement stack, queue or sorted list Example:

Sketchy, singly-linked list can be shown like this:

Each cell is called a node of a singly-linked list. First node is called head and it's a dedicated node. By knowing it, we can access every other node in the list. Sometimes, last node, called tail, is also stored in order to speed up add operation.

### **III. Operations on a singly-linked list**

Concrete implementation of operations on the singly-linked list depends on the purpose, it is used for. Following the links below, you can find descriptions of the common concepts, proper for every implementation.

- Singly-linked list traversal
- Adding a node
- Removing a node

### **a) Singly-linked list. Traversal.**

Assume that we have a list with some nodes. Traversal is the very basic operation, which presents as a part in almost every operation on a singly-linked list. For instance, algorithm may traverse a singly-linked list to find a value, find a position for insertion, etc. For a singly-linked list, only forward direction traversal is possible.

Traversal algorithm

Beginning from the head,

1. check, if the end of a list hasn't been reached yet;
2. do some actions with the current node, which is specific for particular algorithm;
3. Current node becomes previous and next node becomes current. Go to the step 1.

### **Example**

As for example, let us see an example of summing up values in a singly-linked list.

For some algorithms tracking the previous node is essential, but for some, like an example, it's unnecessary. We show a common case here and concrete algorithm can be adjusted to meet its individual requirements.

### **b) Singly-linked list. Addition (insertion) operation.**

Insertion into a singly-linked list has two special cases. It's insertion a new node before the head (to the very beginning of the list) and after the tail (to the very end of the list). In any other case, new node is inserted in the middle of the list and so, has a predecessor and successor in the list. There is a description of all these cases below.

### **Empty list case**

When list is empty, which is indicated by (head == NULL) condition, the insertion is quite simple. Algorithm sets both head and tail to point to the new node.

### **Add first**

In this case, new node is inserted right before the current head node.

It can be done in two steps:

1. Update the next link of a new node, to point to the current head node.
2. Update head link to point to the new node.

### **Add last**

In this case, new node is inserted right after the current tail node.

It can be done in two steps:

1. Update the next link of the current tail node, to point to the new node.
2. Update tail link to point to the new node.

### **General case**

In general case, new node is always inserted between two nodes, which are already in the list. Head and tail links are not updated in this case.

Such an insert can be done in two steps:

1. Update link of the "previous" node, to point to the new node.
2. Update link of the new node, to point to the "next" node.

### **c) Singly-linked list. Removal (deletion) operation.**

There are four cases, which can occur while removing the node. These cases are similar to the cases in add operation. We have the same four situations, but the order of algorithm actions is opposite. Notice, that removal algorithm includes the disposal of the deleted node, which may be unnecessary in languages with automatic garbage collection (i.e., Java).

### **List has only one node**

When list has only one node, which is indicated by the condition, that the head points to the same node as the tail, the removal is quite simple. Algorithm disposes the node, pointed by head (or tail) and sets both head and tail to NULL.

### **Remove first**

In this case, first node (current head node) is removed from the list. It can be done in two steps:

1. Update head link to point to the node, next to the head.
2. Dispose removed node.

### **Remove last**

In this case, last node (current tail node) is removed from the list. This operation is a bit more tricky, than removing the first node, because algorithm should find a node, which is previous to the tail first.

It can be done in three steps:

1. Update tail link to point to the node, before the tail. In order to find it, list should be traversed first, beginning from the head.
2. Set next link of the new tail to NULL.
3. Dispose removed node.

### **General case**

In general case, node to be removed is always located between two list nodes. Head and tail links are not updated in this case.

Such a removal can be done in two steps:

1. Update next link of the previous node, to point to the next node, relative to the removed node.
2. Dispose removed node.

### **d) Singly-linked list. Internal representation.**

Every node of a singly-linked list contains following information:

- a value (user's data);
- a link to the next element (auxiliary data).

Sketchy, it can be shown like this:

First node called head and no other node points to it. Link to the head is usually stored in the class, which provides an interface to the resulting data structure. For empty list, head is set to NULL. Also, it makes sense to store a link to the last node, called tail. Though no node in the list can be accessed from the tail (because we can move forward only), it can accelerate an add operation, when adding to the end of the list. When list is big, it reduces add operation complexity essentially, while memory overhead is insignificant.

Doubly linked list with the help of algorithms for Insertion, Deletion an element

### **IV. Introduction to:**

1. DOUBLY LINKED LIST
2. implementation of DOUBLY LINKED LIST
3. Operation of DOUBLY LINKED LIST

In singly linked list, we can move/traverse only in one single direction because each node has the address of the next node only. Suppose we are in the middle of the linked list and we want the address of previous node then we don't have any option other than repeating the traversing from the beginning node.

To overcome this drawback, a doubly linked list can be used. In this, there are two pointers. One of these pointers points to the next node and the other points to the previous node.

The structure for a doubly linked list node can be declared as:

```
struct node
{
    struct node *prev_node;
    int info;
    struct node *next_node;
};
```

Prev node would contain a pointer to the address of the previous node and next\_node would point the next node in the list. Hence, we can move in both the directions.

## LAB TASK

- 1 Write a program as follows

-----Linked List implementation of LIST ADT-----

1. Insert an element at the end
2. Delete an element from end
3. Search an element
4. Display List
5. Is List Empty
6. Exit

-----  
Please Enter Your Choice:

## Code:

```
#include<iostream>
#include<conio.h>
using namespace std;
class node //class of Node
{
public:
    int data;
    node* next;
    node(int value)
```



## Data Structures and Algorithms (CS-212)

```
        {
            data = value;
            next = NULL;
        }
    };

void InserAtTail(node* &head, int val) //Insert at Tail Function
{
    node* n = new node(val);
    if (head == NULL)
    {
        head = n;
        return;
    }
    node* temp = head;
    while (temp->next != NULL)
    {
        temp = temp->next;
    }
    temp->next = n;
}

void DeletionAtEnd(node* &head) // Delete Function only for 'Tail Or End'
{
    node* temp = head;
    if (temp == NULL)
    {
        cout << "\n \t No data to display because Linked List is Empty " <<
endl;
    }
    else
    {
        node* todelete = temp->next;
        temp->next = temp->next->next;

        delete todelete;
        cout << "\n \t Last Value has been deleted from the Linked List " <<
endl;
    }
}

bool search(node* head, int key) // Searching Function
{
    node* temp = head;
    int count = 0;
    while (temp != NULL)
    {
        count++;
        if (temp->data == key)
        {
            cout << "\n \t Value is present at location : " << count << endl;
            return true;
            break;
        }
        temp = temp->next;
    }
    cout << "\n \t Value is Not Present " << endl;
    return false;
}

void Display(node* head) // For Display Function
{
```

## Data Structures and Algorithms (CS-212)

```
node* temp = head;
cout << "\n \t ";
while (temp != NULL)
{
    cout << temp->data << " -> ";
    temp = temp->next;
}
cout << "NULL" << endl;
}

void Is_linklist_Empty(node* head)    // Linklist checking
{
    if (head == NULL)
    {
        cout << "\n \t No data to display because LinkED List is Empty ...." <<
endl;
    }
    else
    {
        cout << "\n \t There is a data inside linklist ....." << endl;
    }
}

int main()    //main function
{
    node* head = NULL;

    jump:
    char choose;
    cout << "\n \t -----Linked List implementation of LIST ADT-----
-----" << endl;
    cout << "\n \t Enter '1' for Insert an element at the end " << endl;
    cout << " \t Enter '2' for Delete an element from end " << endl;
    cout << " \t Enter '3' for Search an element " << endl;
    cout << " \t Enter '4' for Display List " << endl;
    cout << " \t Enter '5' for Is List Empty ? " << endl;
    cout << " \t Enter '6' for Exit the program " << endl;
    cout << "\n \t Enter the Number : ";
    cin >> choose;

    switch (choose)
    {
        case '1':
        {
            int value;
            cout << "\n \t Enter the Element you want to insert : ";
            cin >> value;
            InsertAtTail(head, value);
            cout << "\n \t Value is Entered " << endl;
            break;
        }
        case '2':
        {
            DeletionAtEnd(head);
            break;
        }
        case '3':
        {
            int s;
            cout << "\n \t Enter the Value you want to Search : ";
            cin >> s;
            search(head, s);
        }
    }
}
```

```

        break;
    }
    case '4':
    {
        Display(head);
        break;
    }
    case '5':
    {
        Is_linklist_Empty(head);
        break;
    }
    case '6':
    {
        exit(0);
        break;
    }
    }
    goto jump;

    system("pause");
    return 0;
}

```

## Output:

```

-----Linked List implementation of LIST ADT-----

Enter '1' for Insert an element at the end
Enter '2' for Delete an element from end
Enter '3' for Search an element
Enter '4' for Display List
Enter '5' for Is List Empty ?
Enter '6' for Exit the program

Enter the Number : 1

Enter the Element you want to insert : 4

Value is Entered

-----Linked List implementation of LIST ADT-----

Enter '1' for Insert an element at the end
Enter '2' for Delete an element from end
Enter '3' for Search an element
Enter '4' for Display List
Enter '5' for Is List Empty ?
Enter '6' for Exit the program

Enter the Number : 1

Enter the Element you want to insert : 7

Value is Entered

-----Linked List implementation of LIST ADT-----

Enter '1' for Insert an element at the end
Enter '2' for Delete an element from end
Enter '3' for Search an element
Enter '4' for Display List
Enter '5' for Is List Empty ?
Enter '6' for Exit the program

Enter the Number : 4

4 -> 7 -> NULL

```

```
-----Linked List implementation of LIST ADT-----

Enter '1' for Insert an element at the end
Enter '2' for Delete an element from end
Enter '3' for Search an element
Enter '4' for Display List
Enter '5' for Is List Empty ?
Enter '6' for Exit the program

Enter the Number : 3

Enter the Value you want to Search : 7

Value is present at location : 2

-----Linked List implementation of LIST ADT-----

Enter '1' for Insert an element at the end
Enter '2' for Delete an element from end
Enter '3' for Search an element
Enter '4' for Display List
Enter '5' for Is List Empty ?
Enter '6' for Exit the program

Enter the Number : 2

Last Value has been deleted from the Linked List

-----Linked List implementation of LIST ADT-----

Enter '1' for Insert an element at the end
Enter '2' for Delete an element from end
Enter '3' for Search an element
Enter '4' for Display List
Enter '5' for Is List Empty ?
Enter '6' for Exit the program

Enter the Number : 4

4 -> NULL
```

```
-----Linked List implementation of LIST ADT-----

Enter '1' for Insert an element at the end
Enter '2' for Delete an element from end
Enter '3' for Search an element
Enter '4' for Display List
Enter '5' for Is List Empty ?
Enter '6' for Exit the program

Enter the Number : 5

There is a data inside linklist .....

-----Linked List implementation of LIST ADT-----

Enter '1' for Insert an element at the end
Enter '2' for Delete an element from end
Enter '3' for Search an element
Enter '4' for Display List
Enter '5' for Is List Empty ?
Enter '6' for Exit the program

Enter the Number : 6
```

# **Lab # 7**

## **An array based implementation of STACK with the help of algorithm**

### **I. Introduction to:**

1. STACK
2. An array-based implementation of STACK
3. Operation of STACK

### **II. Stack**

Stack: is an ordered group of homogeneous items of elements.

Elements are added to and removed from the top of the stack (the most recently added items are at the top of the stack).

The last element to be added is the first to be removed (LIFO: Last In, First Out).

#### **a) Stack Operation**

MakeEmpty

Boolean IsEmpty

Boolean IsFull

Push (ItemType newItem)

Pop (ItemType& item) (or pop and top)

Push (ItemType newItemId)

Function: Adds newItem to the top of the stack.

Preconditions: Stack has been initialized and is not full.

Postconditions: newItem is at the top of the stack.

Pop (Item Type & Item)

Function: Removes top Item from stack and returns it in item.

Preconditions: Stack has been initialized and is not empty.

Postconditions: Top element has been removed from stack and item is a copy of the removed element.

# Lab Tasks

1 Write a program as follows for STACK

-----Array based implementation of STACK-----

1. Push an element on stack
2. Pop an element from stack
3. Display all
4. Top element
5. Exit

-----

Please Enter Your Choice:

2 Test the program using the following procedure: STACK of size N=6

1. Call PUSH(5)
2. Call PUSH(2)
3. Call PUSH(3)
4. Call POP()
5. Call PUSH(6)
6. Call PUSH(9)
7. Call PUSH(3)
8. Call DISPLAY()
9. Call TOP()

## Code:

```
#include<iostream>
#include<conio.h>

using namespace std;
int stack[100], n = 100, top = -1;

void push(int val) // Push Function
{
    if (top >= n - 1)
    {
        cout << "\n \t Stack Overflow" << endl;
```

```

    }
    else
    {
        top++;
        stack[top] = val;
    }
}

void pop() // Pop Function
{
    if (top <= -1)
    {
        cout << "\n \t Stack Underflow" << endl;
    }
    else
    {
        cout << "\n \t The popped element is : " << stack[top] << endl;
        top--;
    }
}

void display() // Display Function
{
    if (top >= 0)
    {
        cout << "\n \t Stack Elements are : ";

        for (int i = top; i >= 0; i--)
        {
            cout << stack[i] << ", ";
        }
        cout << endl;
    }
    else
    {
        cout << "\n \t Stack is empty" << endl;
    }
}

void display_top() // Top Function
{
    cout << "\n \t Top is : " << stack[top];
}

int main() // Main Function
{
loop:
    int value;
    char ch;
    cout << "\n \t ----- Array based implementation of STACK -----
    ----- " << endl;
    cout << "\n \t Enter the '1' for Push In Stack " << endl;
    cout << "\n \t Enter the '2' for Pop In Stack " << endl;
    cout << "\n \t Enter the '3' for Display " << endl;
    cout << "\n \t Enter the '4' for Display the TOP Element " << endl;
    cout << "\n \t Enter the '5' for Exit " << endl;
    cout << "\n \t Enter the Choice : ";
    cin >> ch;
    cout << "\n \t _____ " << endl;
    switch (ch)
    {

```

```

    case '1':
    {
        cout << "\n \t Enter Value to be pushed : ";
        cin >> value;
        push(value);
        break;
    }
    case '2':
    {
        pop();
        break;
    }
    case '3':
    {
        display();
        break;
    }
    case '4':
    {
        display_top();
        break;
    }
    case '5':
    {
        exit(0);
        break;
    }
}
cout << "\n \t _____ " << endl;
goto loop;
system("pause");
return 0;
}

```



# Output:

```
----- Array based implementation of STACK -----  
  
Enter the '1' for Push In Stack  
Enter the '2' for Pop In Stack  
Enter the '3' for Display  
Enter the '4' for Display the TOP Element  
Enter the '5' for Exit  
Enter the Choice : 1
```

---

```
Enter Value to be pushed : 5
```

```
----- Array based implementation of STACK -----  
  
Enter the '1' for Push In Stack  
Enter the '2' for Pop In Stack  
Enter the '3' for Display  
Enter the '4' for Display the TOP Element  
Enter the '5' for Exit  
Enter the Choice : 1
```

---

```
Enter Value to be pushed : 2
```

```
----- Array based implementation of STACK -----  
  
Enter the '1' for Push In Stack  
Enter the '2' for Pop In Stack  
Enter the '3' for Display  
Enter the '4' for Display the TOP Element  
Enter the '5' for Exit  
Enter the Choice : 1
```

---

```
Enter Value to be pushed : 3
```

```
----- Array based implementation of STACK -----  
  
Enter the '1' for Push In Stack  
Enter the '2' for Pop In Stack  
Enter the '3' for Display  
Enter the '4' for Display the TOP Element  
Enter the '5' for Exit  
Enter the Choice : 2
```

---

```
The popped element is : 3
```

```
----- Array based implementation of STACK -----  
  
Enter the '1' for Push In Stack  
Enter the '2' for Pop In Stack  
Enter the '3' for Display  
Enter the '4' for Display the TOP Element  
Enter the '5' for Exit  
Enter the Choice : 1
```

---

```
Enter Value to be pushed : 6
```

---

----- Array based implementation of STACK -----

Enter the '1' for Push In Stack  
Enter the '2' for Pop In Stack  
Enter the '3' for Display  
Enter the '4' for Display the TOP Element  
Enter the '5' for Exit  
Enter the Choice : 1

Enter Value to be pushed : 9

----- Array based implementation of STACK -----

Enter the '1' for Push In Stack  
Enter the '2' for Pop In Stack  
Enter the '3' for Display  
Enter the '4' for Display the TOP Element  
Enter the '5' for Exit  
Enter the Choice : 1

Enter Value to be pushed : 3

----- Array based implementation of STACK -----

Enter the '1' for Push In Stack  
Enter the '2' for Pop In Stack  
Enter the '3' for Display  
Enter the '4' for Display the TOP Element  
Enter the '5' for Exit  
Enter the Choice : 3

Stack Elements are : 3, 9, 6, 2, 5,

----- Array based implementation of STACK -----

Enter the '1' for Push In Stack  
Enter the '2' for Pop In Stack  
Enter the '3' for Display  
Enter the '4' for Display the TOP Element  
Enter the '5' for Exit  
Enter the Choice : 4

Top is : 3

----- Array based implementation of STACK -----

Enter the '1' for Push In Stack  
Enter the '2' for Pop In Stack  
Enter the '3' for Display  
Enter the '4' for Display the TOP Element  
Enter the '5' for Exit  
Enter the Choice :

## **Lab # 8**

### **An array based implementation of QUEUE with the help of algorithm**

#### **I. Introduction to:**

1. QUEUE
2. An array based implementation of QUEUE
3. Operation of QUEUE

#### **II. Queue Data Structures**

Queue is also an abstract data type or a linear data structure, in which the first element is inserted from one end called REAR(also called tail), and the deletion of existing element takes place from the other end called as FRONT(also called head). This makes queue as FIFO data structure, which means that element inserted first will also be removed first.

The process to add an element into queue is called Enqueue and the process of removal of an element from queue is called Dequeue.

##### **a) Basic features of Queue**

Like Stack, Queue is also an ordered list of elements of similar data types.

1. Queue is a FIFO( First in First Out ) structure.
2. Once a new element is inserted into the Queue, all the elements inserted before the new element in the queue must be removed, to remove the new element.
3. peek( ) function is oftenly used to return the value of first element without dequeuing it.

##### **b) Application of Queue**

Queue, as the name suggests is used whenever we need to have any group of objects in an order in which the first one coming in, also gets out first while the others wait for there turn, like in the following scenarios :

Serving requests on a single shared resource, like a printer, CPU task scheduling etc.

1. In real life, Call Center phone systems will use Queues, to hold people calling them in an order, until a service representative is free.
2. Handling of interrupts in real-time systems. The interrupts are handled in the same order as they arrive, First come first served.

### c) Implementation of Queue

Queue can be implemented using an Array, Stack or Linked List. The easiest way of implementing a queue is by using an Array. Initially the head(FRONT) and the tail(REAR) of the queue points at the first index of the array (starting the index of array from 0). As we add elements to the queue, the tail keeps on moving ahead, always pointing to the position where the next element will be inserted, while the head remains at the first index.

When we remove element from Queue, we can follow two possible approaches (mentioned [A] and [B] in above diagram). In [A] approach, we remove the element at head position, and then one by one move all the other elements on position forward. In approach [B] we remove the element from head position and then move head to the next position.

In approach [A] there is an overhead of shifting the elements one position forward every time we remove the first element. In approach [B] there is no such overhead, but whenever we move head one position ahead, after removal of first element, the size on Queue is reduced by one space each time.

### Lab Task

1. Write a program as follows for QUEUE

-----Array based implementation of QUEUE-----

1. Enqueue an element to queue
2. Dequeue an element from queue
3. Display all
4. Exit

-----

Please Enter Your Choice:

2. Test the program using the following procedure: QUEUE of size N=6

Call enqueue(5)

Call enqueue(2)

Call enqueue(3)

Call dequeue()

Call dequeue()

Call enqueue(6)

Call enqueue(3)

Call Display()

## CODE :

```
#include<iostream>
#include<conio.h>
using namespace std;

int queue[50], n = 6, front = -1, rear = -1;

void Enqueue() // Insert Function
{
    int val;
    if (rear == n - 1)
    {
        cout << "\n \t \t Queue Overflow" << endl;
    }
    else
    {
        if (front == -1)
        {
            front = 0;
        }
        cout << "\n \t Insert the element in queue : ";
        cin >> val;
        rear++;
        queue[rear] = val;
    }
}

void Dequeue() // Remove Function
{
    if (front == -1 || front > rear)
    {
        cout << "\n \t \t Queue Underflow ";
        return;
    }
    else
    {
        cout << "\n \t Element deleted from queue is : " << queue[front] << endl;
        front++;
    }
}

void Display() // Display Function
{
    if (front == -1)
    {
        cout << "\n \t \t Queue is empty" << endl;
    }
    else
    {
        cout << "\n \t Queue elements are : ";
        for (int i = front; i <= rear; i++)
        {
            cout << "\n \t Position : " << i << " , Value : " << queue[i];
        }
        cout << endl;
    }
}
```

## Data Structures and Algorithms (CS-212)

```
int main()    // Main Function
{
    int ch;
    cout << "\n \t 1) Insert element to queue" << endl;
    cout << "\n \t 2) Delete element from queue" << endl;
    cout << "\n \t 3) Display all the elements of queue" << endl;
    cout << "\n \t 4) Exit" << endl;

    do
    {
        cout << "\n \t _____ " << endl;
        cout << "\n \t \t Enter your choice : ";
        cin >> ch;
        cout << "\n \t _____ " << endl;
        switch (ch)
        {
            case 1:
            {
                Enqueue();
                break;
            }
            case 2:
            {
                Dequeue();
                break;
            }
            case 3:
            {
                Display();
                break;
            }
            case 4:
            {
                cout << "\n \t \t \t Exit" << endl;
                break;
            }
            default:
            {
                cout << "\n \t \t Invalid choice" << endl;
            }
        }
    } while (ch != 4);

    return 0;
}
```

## OUTPUT :

```
1> Insert element to queue
2> Delete element from queue
3> Display all the elements of queue
4> Exit
```

---

```
Enter your choice : 1
```

---

```
Insert the element in queue : 5
```

---

```
Enter your choice : 1
```

---

```
Insert the element in queue : 2
```

---

```
Enter your choice : 1
```

---

```
Insert the element in queue : 3
```

---

```
Enter your choice : 2
```

---

```
Element deleted from queue is : 5
```

---

```
Enter your choice : 2
```

---

```
Element deleted from queue is : 2
```

---

```
Enter your choice : 1
```

---

```
Insert the element in queue : 6
```

---

```
Enter your choice : 1
```

---

```
Insert the element in queue : 3
```

---

```
Enter your choice : 3
```

---

```
Queue elements are :
Position : 2 , Value : 3
Position : 3 , Value : 6
Position : 4 , Value : 3
```

---

```
Enter your choice :
```

## **Lab # 9**

### **Implementation of the BINARY TREE with the help of algorithms**

#### **I. Introduction to:**

1. Binary Search Tree definition
2. Creation of Binary Search Tree
3. Operation of Binary Search Tree

#### **II. Binary Search Tree**

First of all, binary search tree (BST) is a dynamic data structure, which means, that its size is only limited by amount of free memory in the operating system and number of elements may vary during the program run. Main advantage of binary search trees is rapid search, while addition is quite cheap. Let us see more formal definition of BST.

Binary search tree is a data structure, which meets the following requirements:

- it is a binary tree;
- each node contains a value;
- a total order is defined on these values (every two values can be compared with each other)
- leftsubtree of a node contains only values lesser, than the node's value;
- rightsubtree of a node contains only values greater, than the node's value.

Notice, that definition above doesn't allow duplicates.

#### **a) Example of a binary search tree**

Binary search tree. Internal representation

Like any other dynamic data structure, BST requires storing of some additional auxiliary data, in order to keep its structure. Each node of binary tree contains the following information:



- a value (user's data);
- a link to the left child (auxiliary data);
- a link to the right child (auxiliary data).

Depending on the size of user data, memory overhead may vary, but in general it is quite reasonable. In some implementations, node may store a link to the parent, but it depends on algorithm, programmer want to apply to BST. For basic operations, like addition, removal and search a link to the parent is not necessary. It is needed in order to implement iterators.

With a view to internal representation, the sample from the overview changes:

Leaf nodes have links to the children, but they don't have children. In a programming language it means, that corresponding links are set to NULL.

### **b) Binary search tree. Removing a node**

Remove operation on binary search tree is more complicated, than add and search. Basically, it can be divided into two stages:

- search for a node to remove;
- if the node is found, run remove algorithm.

Remove algorithm in detail

Now, let's see more detailed description of a remove algorithm. First stage is identical to algorithm for lookup, except we should track the parent of the current node. Second part is trickier.

There are three cases, which are described below.

#### **1. Node to be removed has no children.**

This case is quite simple. Algorithm sets corresponding link of the parent to NULL and disposes the node.

Example. Remove -4 from a BST.

#### **2. Node to be removed has one child.**

In this case, node is cut from the tree and algorithm links single child (with its subtree) directly to the parent of the removed node. Example.

Remove 18 from a BST.

3. Node to be removed has two children.

This is the most complex case. To solve it, let us see one useful BST property first. We are going to use the idea, that the same set of values may be represented as different binary-search trees. For example, those BSTs:

contains the same values {5, 19, 21, 25}. To transform first tree into second one, we can do following:

choose minimum element from the right subtree (19 in the example);

replace 5 by 19;      hang 5 as a left child.

The same approach can be utilized to remove a node, which has two children:

find a minimum value in the right subtree;

replace value of the node to be removed with found minimum.

Now, right subtree contains a duplicate!

apply remove to the right subtree to remove a duplicate.

Notice, that the node with minimum value has no left child and, therefore, it's removal may result in first or second cases only.

Example. Remove 12 from a BST.

Find minimum element in the right subtree of the node to be removed. In current example it is 19.

Replace 12 with 19. Notice, that only values are replaced, not nodes. Now we have two nodes with the same value.

Remove 19 from the left subtree.

## Lab Task

1. Write a program to create a Tree for the following.

Insert Sequence of data: 52, 25, 60, 35, 20, 59, 65

### Program:

```
#include<iostream>
using namespace std;

struct Node    //Definition of Node for Binary search tree
{
    int data;
    Node *left, *right;
    Node(int val)
    {
        data = val;
        left = NULL;
        right = NULL;
    }
};

// To insert data in BST, returns address of root node
Node* Insert(Node* root, int val)
{
    if (root == NULL) // empty tree
    {
        return new Node(val);
    }
    // if data to be inserted is lesser, insert in left subtree.
    else if (val < root->data)
    {
        root->left = Insert(root->left, val); //updating
    }
    // else, insert in right subtree.
    else
    {
        root->right = Insert(root->right, val);
    }
    return root;
}

//To search an element in BST, returns true if element is found
bool Search(Node* root, int data)
{
    if (root == NULL)
    {
        return false;
    }
    else if (root->data == data)
    {
        return true;
    }
    else if (data <= root->data)
    {
        return Search(root->left, data);
    }
}
```

```
        else
        {
            return Search(root->right, data);
        }
    }

// to print the elements in Sorted Order
void inorder(Node* root)
{
    if (root == NULL)
    {
        return;
    }
    inorder(root->left);
    cout << root->data << " ";
    inorder(root->right);
}

int main()
{
    Node* root = NULL; // Creating an empty tree

    /*Code to test the logic*/
    root = Insert(root, 52);
    root = Insert(root, 25);
    root = Insert(root, 60);
    root = Insert(root, 35);
    root = Insert(root, 20);
    root = Insert(root, 59);
    root = Insert(root, 65);

    cout << "\n \t We have inserted : 52, 25, 60, 35, 20, 59, 65 " << endl;

    // print inorder
    cout << "\n \t After inorder we get : ";
    inorder(root);
    cout << endl;

    // Ask user to enter a number.
    int number;
    cout << "\n \t Enter number be searched : ";
    cin >> number;

    // If number is found, print "FOUND"
    cout << "\n \t _____ " << endl;
    if (Search(root, number) == true)
    {
        cout << "\n \t \t Found" << endl;
    }
    else
    {
        cout << "\n \t \t Not Found" << endl;
    }
    cout << " \t _____ " << endl;

    system("pause");
    return 0;
}
```

## Output:

```
We have inserted : 52, 25, 60, 35, 20, 59, 65
After inorder we get : 20, 25, 35, 52, 59, 60, 65,
Enter number be searched : 60
```

---

```
Found
```

---

```
Press any key to continue . . .
```

```
We have inserted : 52, 25, 60, 35, 20, 59, 65
After inorder we get : 20, 25, 35, 52, 59, 60, 65,
Enter number be searched : 7
```

---

```
Not Found
```

---

```
Press any key to continue . . .
```

# **Lab # 10**

## **Implementation of the BINARY SEARCH TREE data structure with the help of algorithms**

### **I. Introduction to:**

1. BINARY SEARCH TREE structure representation
2. Creation of Binary Search Tree
3. Traversing over Binary Search Tree

### **II. Binary Tree**

We extend the concept of linked data structures to structure containing nodes with more than one self-referenced field. A binary tree is made of nodes, where each node contains a "left" reference, a "right" reference, and a data element. The topmost node in the tree is called the root.

Every node (excluding a root) in a tree is connected by a directed edge from exactly one other node. This node is called a parent. On the other hand, each node can be connected to arbitrary number of nodes, called children. Nodes with no children are called leaves, or external nodes. Nodes which are not leaves are called internal nodes. Nodes with the same parent are called siblings.

#### **a) Binary Search Tree**

First of all, binary search tree (BST) is a dynamic data structure, which means, that its size is only limited by amount of free memory in the operating system and number of elements may vary during the program run. Main advantage of binary search trees is rapid search, while addition is quite cheap. Let us see more formal definition of BST.

Binary search tree is a data structure, which meets the following requirements:

- it is a binary tree;
- each node contains a value;
- a total order is defined on these values (every two values can be compared with each other);
- leftsubtree of a node contains only values lesser, than the node's value;
- rightsubtree of a node contains only values greater, than the node's value. Notice, that definition above doesn't allow duplicates.

### Example of a binary search tree

#### Binary search tree. Internal representation

Like any other dynamic data structure, BST requires storing of some additional auxiliary data, in order to keep its structure. Each node of binary tree contains the following information:

- a value (user's data);
- a link to the left child (auxiliary data);
- a link to the right child (auxiliary data).

Depending on the size of user data, memory overhead may vary, but in general it is quite reasonable. In some implementations, node may store a link to the parent, but it depends on algorithm, programmer want to apply to BST. For basic operations, like addition, removal and search a link to the parent is not necessary. It is needed in order to implement iterators.

With a view to internal representation, the sample from the overview changes:

Leaf nodes have links to the children, but they don't have children. In a programming language it means, that corresponding links are set to NULL.

# **Lab # 11**

## **Implementation Tree Traversal with the help of algorithms**

### **I. Tree Traversal:**

A traversal is a process that visits all the nodes in the tree. Since a tree is a nonlinear data structure, there is no unique traversal. We will consider several traversal algorithms with we group in the following two kinds

- depth-first traversal
- breadth-first traversal

There are three different types of depth-first traversals, :

- PreOrder traversal - visit the parent first and then left and right children;
- InOrder traversal - visit the left child, then the parent and the right child;
- PostOrder traversal - visit left child, then the right child and then the parent;

There is only one kind of breadth-first traversal--the level order traversal. This traversal visits nodes by levels from top to bottom and from left to right.

As an example consider the following tree and its four traversals:

PreOrder - 8, 5, 9, 7, 1, 12, 2, 4, 11, 3

InOrder - 9, 5, 1, 7, 2, 12, 8, 4, 3, 11

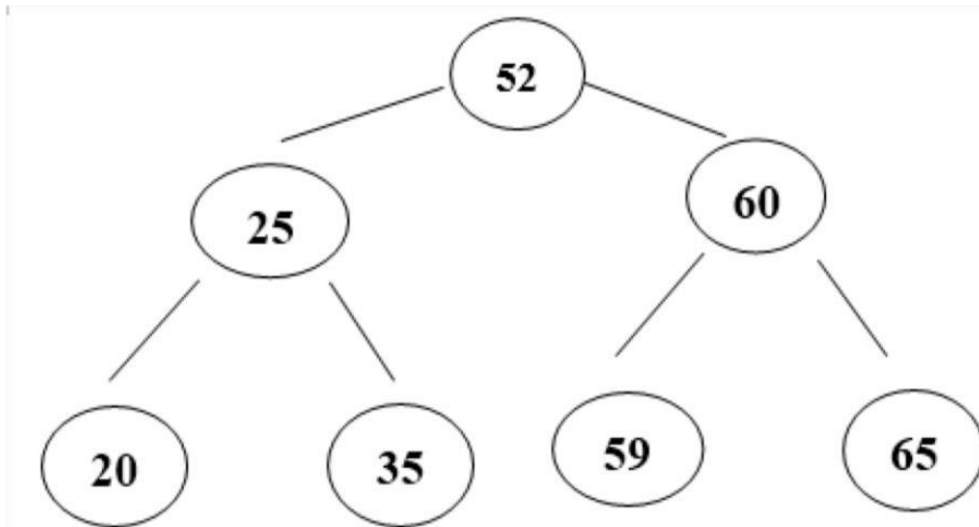
PostOrder - 9, 1, 2, 12, 7, 5, 3, 11, 4, 8

LevelOrder - 8, 5, 4, 9, 7, 11, 1, 12, 3, 2



## Lab Task

1. Write a program to print the nodes of the following Tree diagram through Pre-Order, InOrder and Post-Order.



### Program :

```
#include<iostream>
using namespace std;

struct Node
{
    int data;
    struct Node* left;
    struct Node* right;
};

/*PreOrder traversal - visit the parent first and then left and right children;*/

void Preorder(Node* root)      // Pre-Order Function
{
    if (root == NULL)
    {
        return;
    }
    cout << root->data << ", ";
    Preorder(root->left);
    Preorder(root->right);
}

/*InOrder traversal - visit the left child, then the parent and the right child;*/

void Inorder(Node* root)      // In-Order Function
{
    if (root == NULL)
    {
        return;
    }
    Inorder(root->left);
    cout << root->data << ", ";
    Inorder(root->right);
}
```

```
/*PostOrder traversal - visit left child, then the right child and then the parent;*/
```

```
void Postorder(Node* root)      // Post-Order Function
```

```
{
    if (root == NULL)
    {
        return;
    }
    Postorder(root->left);
    Postorder(root->right);
    cout << root->data << ", ";
}
```

```
Node* Insert(Node* root, int data)    //Insert Function of Data
```

```
{
    if (root == NULL)
    {
        root = new Node();
        root->data = data;
        root->left = root->right = NULL;
    }
    else if (data <= root->data)
    {
        root->left = Insert(root->left, data);
    }
    else
    {
        root->right = Insert(root->right, data);
    }
    return root;
}
```

```
int main()                // Main Function
```

```
{
    Node* root = NULL;
    root = Insert(root, 52);
    root = Insert(root, 25);
    root = Insert(root, 60);
    root = Insert(root, 20);
    root = Insert(root, 35);
    root = Insert(root, 59);
    root = Insert(root, 65);

    cout << "\n \t _____ " << endl;
    cout << "\n \t We have Inserted : 52, 25, 60, 20, 35, 59, 65" << endl;
    cout << "\n \t _____ " << endl;

    cout << "\n \t Preorder : ";
    Preorder(root);

    cout << "\n \n \t Inorder : ";
    Inorder(root);

    cout << "\n \n \t Postorder : ";
    Postorder(root);
    cout << "\n \t _____ " << endl;

    system("pause");
    return 0;
}
```

## Output :

---

```
We have Inserted : 52, 25, 60, 20, 35, 59, 65
```

---

```
Preorder : 52, 25, 20, 35, 60, 59, 65,
```

```
Inorder : 20, 25, 35, 52, 59, 60, 65,
```

```
Postorder : 20, 35, 25, 59, 65, 60, 52,
```

---

```
Press any key to continue . . .
```

## Lab # 12

### Implementation of Graph and its representation

A Graph is a non-linear data structure consisting of nodes and edges. The nodes are sometimes also referred to as vertices and the edges are lines or arcs that connect any two nodes in the graph.

A Graph consists of a finite set of vertices(or nodes) and set of Edges which connect a pair of nodes.

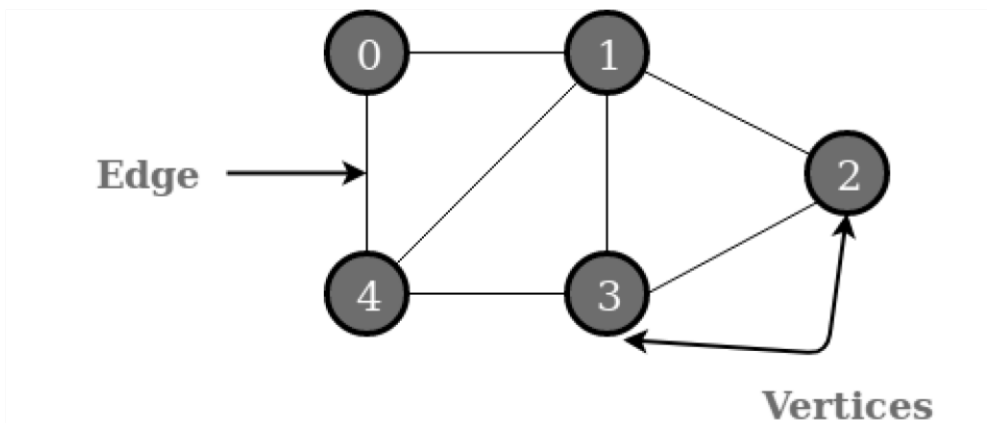


Fig. 2 Graph Representation

In the above Graph, the set of vertices  $V = \{0,1,2,3,4\}$  and the set of edges  $E = \{01, 12, 23, 34, 04, 14, 13\}$ . An example of an undirected graph with 5 vertices.

Graphs are used to solve many real-life problems. Graphs are used to represent networks. The networks may include paths in a city or telephone network or circuit network. Graphs are also used in social networks like LinkedIn, Facebook. For example, in Facebook, each person is represented with a vertex (or node). Each node is a structure and contains information like person id, name, gender, locale etc.

Graph is a data structure that consists of following two components:

1. A finite set of vertices also called as nodes.
2. A finite set of ordered pair of the form  $(u, v)$  called as edge. The pair is ordered because  $(u, v)$  is not same as  $(v, u)$  in case of a directed graph(di-graph). The pair of the form  $(u, v)$  indicates that there is an edge from vertex  $u$  to vertex  $v$ . The edges may contain weight/value/cost.

Following two are the most commonly used representations of a graph.

1. Adjacency Matrix
2. Adjacency List

There are other representations also like, Incidence Matrix and Incidence List. The choice of the graph representation is situation specific. It totally depends on the type of operations to be performed and ease of use.

## I. Adjacency Matrix:

Adjacency Matrix is a 2D array of size  $V \times V$  where  $V$  is the number of vertices in a graph. Let the 2D array be  $adj[i][j]$ , a slot  $adj[i][j] = 1$  indicates that there is an edge from vertex  $i$  to vertex  $j$ . Adjacency matrix for undirected graph is always symmetric. Adjacency Matrix is also used to represent weighted graphs. If  $adj[i][j] = w$ , then there is an edge from vertex  $i$  to vertex  $j$  with weight  $w$ .

The adjacency matrix for the above example graph is:

|   | 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|---|
| 0 | 0 | 1 | 0 | 0 | 1 |
| 1 | 1 | 0 | 1 | 1 | 1 |
| 2 | 0 | 1 | 0 | 1 | 0 |
| 3 | 0 | 1 | 1 | 0 | 1 |
| 4 | 1 | 1 | 0 | 1 | 0 |

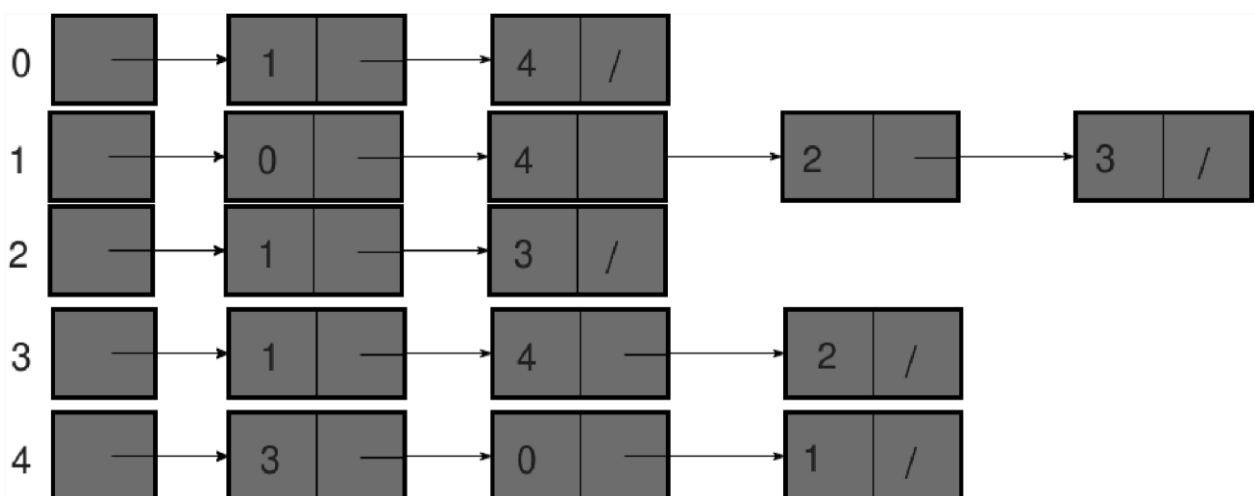
*Pros:* Representation is easier to implement and follow. Removing an edge takes  $O(1)$  time. Queries like whether there is an edge from vertex „ $u$ “ to vertex „ $v$ “ are efficient and can be done  $O(1)$ .

*Cons:* Consumes more space  $O(V^2)$ . Even if the graph is sparse (contains less number of edges), it consumes the same space. Adding a vertex is  $O(V^2)$  time.

Please see [this](#) for a sample Python implementation of adjacency matrix.

## II. Adjacency List:

An array of lists is used. Size of the array is equal to the number of vertices. Let the array be  $array[]$ . An entry  $array[i]$  represents the list of vertices adjacent to the  $i$ th vertex. This representation can also be used to represent a weighted graph. The weights of edges can be represented as lists of pairs. Following is adjacency list representation of the above graph.



# Lab Task

1. Implement representation of graph by Adjacency Matrix and Adjacency List.

## Code:

```
#include <iostream>
#include <conio.h>
#include <list>

using namespace std;

//2D Matrix
int AdjMatrix[50][50];

//This function will print Adjacency Matrix
void PrintAdjMatrix(int v)
{
    cout << "\n _____ " << endl;
    cout << "\n \t Adjacency Matrix for undirected Graph : " << endl;
    cout << "\n _____ " << endl;

    for (int i = 0; i < v; i++)
    {
        for (int j = 0; j < v; j++)
        {
            cout << AdjMatrix[i][j] << " ";
        }
        cout << endl;
    }
}

//This function will mark the vertices which have edges between them
void Edge(int x, int y)
{
    AdjMatrix[x][y] = AdjMatrix[y][x] = 1;
}

//Separate class for Adjacency List
class Graph
{
    int v, e;
    list<int>* l;
public:
    //constructor to pass values from main
    Graph(int v, int e)
    {
        this->v = v;
        this->e = e;
        l = new list<int>[v];
    }

    //This function will push element into a list from back
    void AddEdge(int x, int y)
    {
        l[x].push_back(y);
        l[y].push_back(x);
    }

    //This function will print Adjacency List
}
```

```

void PrintAdjList(int v)
{
    cout << "\n _____" << endl;
    cout << "\n \t Adjacency List for undirected Graph : " << endl;
    cout << "\n _____" << endl;

    for (int i = 0; i < v; i++)
    {
        cout << i;
        for (auto nbr : l[i])
        {
            cout << "-->" << nbr;
        }
        cout << endl;
    }
}

};

//Driver code
void main()
{
    int v, e, x, y;

    //asking user to enter number of vertices
    cout << "\n \t _____" << endl;
    cout << "\n \t Enter the number of vertices in Graph : ";
    cin >> v;

    //asking user to enter number of edges
    cout << "\n \t Enter the number of edges in Graph : ";
    cin >> e;
    cout << "\n \t _____" << endl;

    //line gap
    cout << endl;

    //passing value through constructor
    Graph g(v, e);

    //taking input of edges
    for (int i = 0; i < e; i++)
    {
        cout << "\n \t Enter edge number " << (i + 1) << " : ";
        cin >> x >> y;

        //calling edge function of adjacency matrix
        Edge(x, y);

        //calling edge function of adjacency list
        g.AddEdge(x, y);
    }

    //line gap
    cout << endl;

    //calling function to print Adjacency Matrix
    PrintAdjMatrix(v);

    //line gap
    cout << endl;

    //calling function to print Adjacency List

```

```

        g.PrintAdjList(v);
        cin.get();
    }

```

## Output:

```

Enter the number of vertices in Graph : 5
Enter the number of edges in Graph : 7

Enter edge number 1 : 0 1
Enter edge number 2 : 1 4
Enter edge number 3 : 1 2
Enter edge number 4 : 1 3
Enter edge number 5 : 3 4
Enter edge number 6 : 3 2
Enter edge number 7 : 4 0

Adjacency Matrix for undirected Graph :

0 1 0 0 1
1 0 1 1 1
0 1 0 1 0
0 1 1 0 1
1 1 0 1 0

Adjacency List for undirected Graph :

0-->1-->4
1-->0-->4-->2-->3
2-->1-->3
3-->1-->4-->2
4-->1-->3-->0

```



## Lab # 13

# Implementation of Breadth first and Depth first Traversal for a Graph

### I. Breadth First Traversal (or Search)

Graphs may contain cycles, so we may come to the same node again. To avoid processing a node more than once, we use a boolean visited array. For simplicity, it is assumed that all vertices are reachable from the starting vertex.

For example, in the following graph, we start traversal from vertex 2. When we come to vertex 0, we look for all adjacent vertices of it. 2 is also an adjacent vertex of 0. If we don't mark visited vertices, then 2 will be processed again and it will become a non-terminating process. A Breadth First Traversal of the following graph is 2, 0, 3, 1.

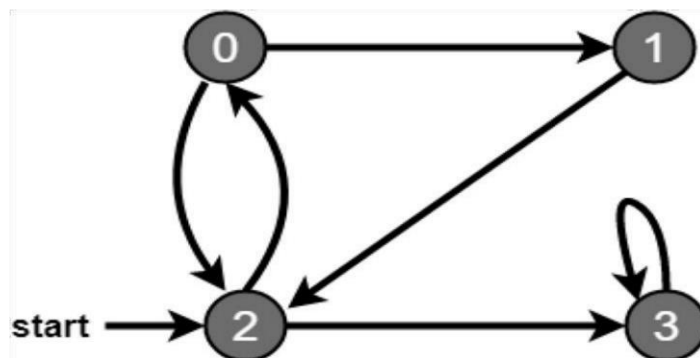


Fig. 3 Graph (Breadth First Traversal)

### II. Depth First Traversal (or Search)

Graphs may contain cycles, so we may come to the same node again. To avoid processing a node more than once, we use a boolean visited array.

For example, in the following graph, we start traversal from vertex 2. When we come to vertex 0, we look for all adjacent vertices of it. 2 is also an adjacent vertex of 0. If we don't mark visited vertices, then 2 will be processed again and it will become a non-terminating process. A Depth First Traversal of the following graph is 2, 0, 1, 3.

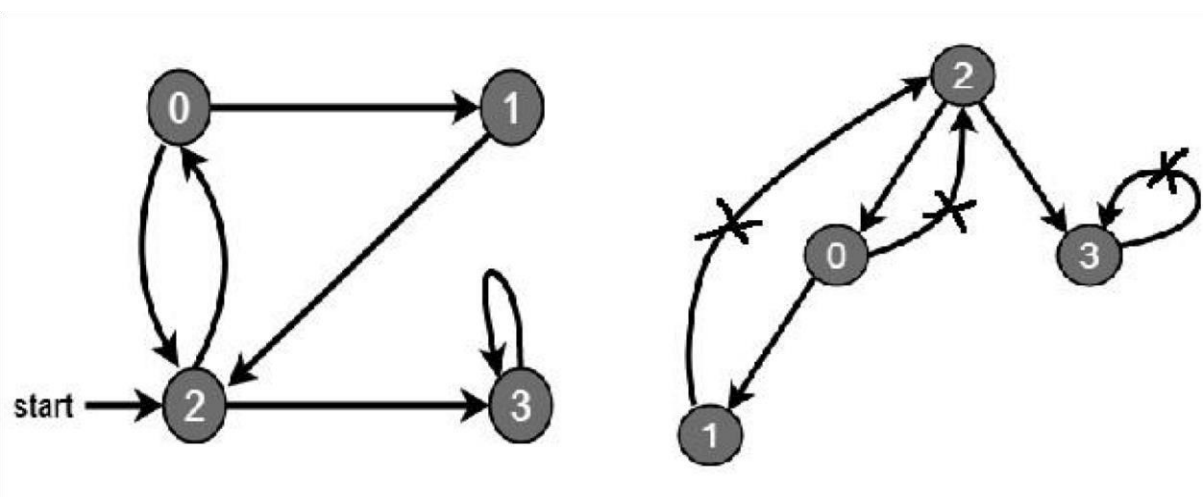


Fig. 4 Graph (Depth First Traversal)

# Lab Task

1. Implement Breadth first and Depth first Traversal for a Graph.

## Code :

```
#include <iostream>
#include <conio.h>
#include <list>
using namespace std;

class Graph
{
    int V;
    list<int>* l;
public:
    //constructor to pass value of vector from main
    Graph(int V)
    {
        this->V = V;
        l = new list<int>[V];
    }

    //This function will push element into a list from back
    void AddEdge(int x, int y)
    {
        l[x].push_back(y);
    }

    //This function will print Breadth First Traversal
    void BreadthFirstTraversal(int S)
    {
        bool* visited = new bool[V];
        //making all vertices not visited
        for (int i = 0; i < V; i++)
        {
            visited[i] = false;
        }

        //creating a list of queue
        list<int>queue;

        //pushing the vertex given by user at the back of queue
        visited[S] = true;
        queue.push_back(S);
        list<int>::iterator i;

        //this loop will run until queue is not empty
        while (!queue.empty())
        {
            //printing the visited vertex in queue
            S = queue.front();
            cout << S << " ";
            queue.pop_front();

            /*pushing the adjacent nodes of vertex which are not in
            the visited list to the back of queue*/
            for (i = l[S].begin(); i != l[S].end(); i++)
            {
                if (!visited[*i])
                {
```

```

        visited[*i] = true;
        queue.push_back(*i);
    }
}

}

void DFS_visit(int d, bool visited[])
{
    //adding the top vertex to visiting list
    visited[d] = true;
    cout << d << " ";
    list<int>::iterator i;

    /*This function will check the top vertex's adjacent vertex
    which are not in the visited list and then prrinting that
    vertex through recursive function*/
    for (i = l[d].begin(); i != l[d].end(); i++)
    {
        if (!visited[*i])
        {
            DFS_visit(*i, visited);
        }
    }
}

/*This function will provide the top vertex and it will
mark all vertices not visited*/
void DepthFirstTraversal(int d)
{
    bool* visited = new bool[V];

    //Mark all the vertices as not visited
    for (int i = 0; i < V; i++)
    {
        visited[i] = false;
    }
    DFS_visit(d, visited);
}

};
//Driver code
void main()
{
    int v, e, x, y, k;
    cout << "\n
\t_____ " << endl;

    //asking user to enter number of vertices
    cout << "\n \t \t \t Enter the number of vertices in Graph : ";
    cin >> v;

    //asking user to enter number of edges
    cout << "\n \t \t \t Enter the number of edges in Graph : ";
    cin >> e;
    cout << "\n
\t_____ " << endl;

    //line gap
    cout << endl;

    //passing value through constructor
    Graph g(v);

    //taking input of edges
    for (int i = 0; i < e; i++)

```

```

{
    cout << "\n \t \t Enter edge number " << (i + 1) << " : ";
    cin >> x >> y;

    //calling edge function to add edges in list
    g.AddEdge(x, y);
}

//line gap
cout << endl;

//asking user to enter starting vertex
cout << "\n
\t_____ " << endl;

    cout << "\n \t \t Enter source vertex : ";
    cin >> k;

    //Printing Breadth First Traversal
    cout << "\n
\t_____ " << endl;

    cout << "\n \t Breadth First Traversal of given Graph is : ";
    g.BreadthFirstTraversal(k);

    //line gap
    cout << endl;

    //Printing Depth First Traversal
    cout << "\n
\t_____ " << endl;

    cout << " \n \t Depth First Traversal of given Graph is : ";
    g.DepthFirstTraversal(k);
    cout << "\n
\t_____ " << endl;

    cout << endl;
    cin.get();
}

```

## Output :

```

Enter the number of vertices in Graph : 4
Enter the number of edges in Graph : 6

Enter edge number 1 : 0 2
Enter edge number 2 : 2 0
Enter edge number 3 : 2 3
Enter edge number 4 : 3 3
Enter edge number 5 : 0 1
Enter edge number 6 : 1 2

Enter source vertex : 2

Breadth First Traversal of given Graph is : 2 0 3 1

Depth First Traversal of given Graph is : 2 0 1 3

```